

README Explanation Document

Companion file to `README.md` — this document explains the structure, content, and design decisions of the Credit Risk Production System README. It is written for readers who want to *understand* the documentation, not just view it.

1. Purpose of the New README

The README was redesigned from a long, linear technical document into a **navigation-first project page** suitable for a GitHub repository. The goals were:

1. **Scannability** — a visitor should understand what the project is, how it performs, and how to use it within 30 seconds.
 2. **Progressive disclosure** — deep technical detail exists, but hidden behind collapsible sections so the main page stays clean.
 3. **Production credibility** — badges, tables, and diagrams signal a mature, production-oriented system.
 4. **Separation of concerns preserved** — the README reflects the platform's core principle: **ML predicts · Policy decides · LLM explains**.
-

2. Section-by-Section Explanation

2.1 Badge Header

The badge row at the top serves three purposes:

Purpose	Explanation
Instant tech-stack recognition	Each badge names a core technology (FastAPI, MLflow, Docker, Kubernetes, etc.) so engineers immediately know the stack.
Visual credibility	Badges are the de-facto standard on professional GitHub projects; their absence can make a repo look unfinished.
Version signaling	Versions shown (e.g., PostgreSQL 15, Python 3.10+) communicate compatibility expectations.

⚠ **Current limitation:** These are *static* badges. Once CI/CD is running (Phase 7), replace them with live badges (build status, test coverage, license, release version) that update automatically.

2.2 Quick Navigation Links

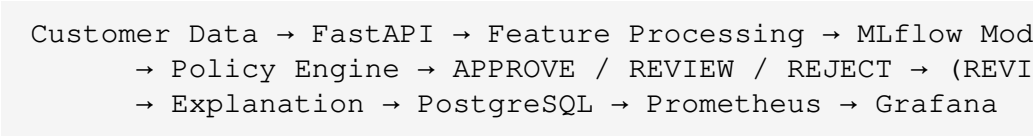
The centered link row (Architecture • Performance • API • Roadmap) jumps directly to the four sections visitors care about most. GitHub renders these as anchor links — no JavaScript needed.

2.3 Production Goal Section

This section compresses the original document's "Production Goal" chapter into a **3-row table**. Why a table?

- Tables force conciseness — each area (ML Engineering, MLOps, LLMOps) gets exactly one line.
 - The **core principle callout** (blockquote) is the single most important sentence in the entire README. It defines the system's philosophy and appears again at the bottom of the page for reinforcement.
-

2.4 Architecture (Mermaid Diagram)



Why Mermaid instead of an image?

Aspect	Mermaid	Static Image
Maintenance	Editable as text in the repo	Requires re-drawing
Dark/light mode	Adapts automatically	Fixed colors
Version control	Diffable in pull requests	Binary blob
Accessibility	Screen-reader friendly	Not accessible

The diagram deliberately shows the **MANUAL_REVIEW** → **RAG** → **LLM** branch, because that is the part newcomers most often misunderstand: the LLM is *only* invoked for review cases, not for every prediction.

Tool Responsibilities (collapsible)

The original document had a 12-row tool table. It was moved into a `<details>` block because:

- It is **reference material**, not introductory material.
- GitHub natively supports `<details>/<summary>` tags in markdown.
- Readers who need it can expand it; readers who don't are not forced to scroll past it.

The same reasoning applies to all other collapsible sections (MLflow lifecycle, JSON example, CI/CD structure, project structure, engineer scopes).

2.5 Model Performance Table

This table is the **quantitative proof** of the project. Design decisions:

1. **Rank column with medals ()** — immediately communicates which model leads without reading numbers.
2. **Status column** — Gradient Boosting is explicitly marked  `Production candidate` while others are  `Registered`, reinforcing the MLflow registry concept from the original document.
3. **Warning blockquote (△)** — the original document stated that the unusually high metrics (99.5%+) must be validated. This warning is kept *directly above the table* so no reader mistakes these numbers as final production evidence.

Below the table, the selection criteria list reminds readers that accuracy alone does not choose a production model.

2.6 API Endpoints Table

The endpoint table answers: *"What can I call?"* Each endpoint has exactly one line. Two design choices:

- `POST` vs `GET` is shown explicitly — important because `/predict`, `/risk/score`, and `/risk/decision` all accept request bodies.
- The **example JSON response** is inside a collapsible block because it is useful reference material but too verbose for the main flow.

The note under the JSON — *"The ML score remains the authoritative numerical prediction"* — is repeated deliberately. It is the most common architectural mistake in ML+LLM systems, so the README states it twice.

2.7 Decision Engine ASCII Diagram

Kept as a simple ASCII diagram rather than Mermaid to provide **visual variety** on the page. It shows the three-way split (`AUTO_APPROVE` / `MANUAL_REVIEW` / `AUTO_REJECT`) and the note that this separation makes decisions *auditable* — a key requirement for credit-risk systems (regulatory expectation, not just engineering preference).

2.8 Observability Matrix

A 4-layer table mapping **API** / **ML** / **Decision** / **LLM** to their metrics. This mirrors the Prometheus & Grafana architecture: every layer of the system produces metrics, and every metric has a defined consumer (the engineering dashboard).

2.9 Development Roadmap

The roadmap uses GitHub **task list syntax** (`- [x]` / `- [ ]`), which GitHub renders as interactive checkboxes. This provides:

- **Progress visibility** — anyone visiting the repo can see Phase 1 is complete and Phase 2 is next.
 - **Built-in tracking** — checking items off in future commits automatically updates the visual state; no external tool needed.
 - **Milestone framing** — each phase bundles related work, matching the original document's Phase 1–7 plan exactly.
-

2.10 Configuration Section

Shows the four required environment variables as a `bash` block. Security principles demonstrated:

- Values are `...` — **never** real secrets.
 - A note states these must come from environment variables / deployment secrets, not Git.
-

2.11 Engineering Scope (Collapsible)

The three roles (ML Engineer, MLOps Engineer, LLMOps Engineer) are each in their own collapsible block. This mirrors the original document's chapters 22–24 but makes the page shorter. Each block states the role's **objective adjectives** (e.g., *reproducible, deployable, observable*) because those are the measurable outcomes, not just task lists.

3. ↺ How the Two Documents Relate

Document	Role	Audience
README.md	Showcase & entry point — what the system is, what it does, how it's doing	Visitors, recruiters, new team members
README_explanation.md (this file)	Rationale & guide — why the README is structured this way, what each choice means	Maintainers, technical writers, future contributors
Original workflow document	Full technical specification — complete detail on every component	Engineers implementing the system

Rule of thumb: if someone asks "*what does this system do?*" → point them to README.md. If they ask "*why is the README organized this way?*" or "*how should I update it?*" → point them to this file.

4. Maintenance Guide

When updating the README, follow these conventions to keep it consistent:

1. **New endpoints** → add a row to the API table, keep descriptions under 10 words.
 2. **New models** → add a row to the performance table, keep the rank/status format.
 3. **New phases** → add a task list item to the roadmap, never remove completed ones (check them off instead — history matters).
 4. **New diagrams** → prefer Mermaid for flows; ASCII is acceptable for small splits.
 5. **Long reference content** → always inside a `<details>` block, never inline.
 6. **Any change to metrics** → the  validation warning must stay until the results are confirmed trustworthy.
-

5. Future Improvements (Post Phase 7)

- ☐ Replace static badges with live CI/CD, coverage, and license badges
- ☐ Add a **"Quick Start"** section with `docker-compose up` instructions once Docker packaging is complete
- ☐ Embed a real Grafana dashboard screenshot under Observability
- ☐ Add a live API demo link (e.g., Swagger UI at `/docs`) once deployed
- ☐ Add a `CONTRIBUTING.md` link once the repo opens to collaborators
- ☐ Consider a **Russian doll structure**: root README links to per-module READMEs (`app/api/README.md`, etc.) as the codebase grows